Linux — Complete Study Notes

Source: Udemy Course Transcript | Trainee: Allen | Environment: Cognizant DevOps / Azure | Goal: Interview Readiness + Interim Assessment

Learning Flow: 6 Phases | Phase 1: Mental Model | Phase 2: Core Building Blocks | Phase 3: Syntax & Configuration | Phase 4: Workflow | Phase 5: Real-World Structure | Phase 6: Interview Favourites

PHASE 1 — The Mental Model: Why Linux Exists and What It Is

1.1 What is an Operating System?

An operating system (OS) is a collection of software that manages hardware resources and provides an environment where applications can run. It allows applications to store information, send documents to a printer, and interact with users. Think of the OS as the manager of a building — it does not do the actual work of the tenants (applications), but it controls who gets electricity, water, and space.

1.2 What is Linux?

When someone says "Linux" they are usually referring to the Linux operating system as a whole. However, technically the term "Linux" refers specifically to the Linux kernel. The kernel is the core or heart of the operating system — the layer that sits between the hardware and the applications running on top of it. Think of the kernel as the engine of a car. The engine alone cannot take you anywhere — you need seats, a steering wheel, doors, and a fuel system. Similarly, the kernel alone cannot be a useful OS — you also need system libraries, a graphical user interface, email utilities, web browsers, and other programs.

1.3 What is a Linux Distribution (Distro)?

A Linux distribution is the Linux kernel bundled together with a collection of software — libraries, utilities, graphical tools, package managers, and applications — packaged and distributed as a single ready-to-use operating system. Think of a Linux distribution like a smartphone model. Different brands (Samsung, Apple, OnePlus) all run on the same underlying idea (a mobile OS), but each one comes with its own default apps, interface style, and bundled software. You can still install your own apps on any of them. The software that comes with a distribution by default

can always be changed — if you do not like the default web browser, you can install another one.

You will often hear the word "distro" as a shorthand for distribution, and sometimes the word "flavour" is used as well.

1.4 The Two Most Important Distros for a DevOps Career

Forget the hundreds of Linux distributions that exist. For a professional DevOps and cloud career, you need to know only two:

Distribution	Use Case / Where You Will Find It	Package Format	Free Alternative
Red Hat Enterprise Linux (RHEL) Pronounced: "REL"	Large traditional organisations: banks, hospitals, telecom companies, government agencies, retailers	RPM	AlmaLinux, Rocky Linux
Ubuntu	Startups, SaaS companies, social networking sites, small e-commerce, cloud-heavy companies	DEB	Ubuntu itself is free

RHEL requires a paid licence, but Red Hat currently allows individuals to run up to 16 systems for free. AlmaLinux and Rocky Linux are drop-in replacements for RHEL — they are literally RHEL with the branding and logos replaced. CentOS used to fill this role but changed to become "CentOS Stream" which is now a preview of upcoming RHEL features, not a stable RHEL clone.

⚠ NOTE: Ubuntu is the correct choice for Azure-based DevOps environments. Most Azure VMs in DevOps pipelines run Ubuntu. AlmaLinux/RHEL knowledge is important for enterprise clients.

1.5 Why Linux for Servers and DevOps?

The command line interface on Linux is far more powerful than a graphical interface for server work. Consider this: if you have 100 files that need renaming, at the command line you can do this with a single command. With a GUI, you would click on each file one by one, 100 times. Most server editions of Linux do not even include a graphical interface — they are designed to be administered entirely from the command line over SSH. This is why every DevOps engineer must be fluent in the Linux command line.

1.6 The Shell — Your Interface to Linux

The shell is the default user interface to a Linux system. When you log in over the network via SSH, the shell is the program that starts and gives you a prompt. When you use a graphical desktop and open a terminal window, you are also accessing the shell. The shell is a command line interpreter — it takes the commands you type, interprets them, and executes them.

The shell prompt gives you information about your session. A typical prompt looks like: jason@linuxsvr:~$ — the dollar sign ($) at the end means you are a normal user. If you see a pound sign (#) at the end, you are the root (superuser) account.

⚠ NOTE: The root user is Linux's equivalent of Windows Administrator. Root can do anything on the system — which is exactly why you should not use root for everyday tasks. Use a normal account and escalate to root only when needed.

1.7 Phase 1 Summary Table

Concept	What It Is	Key Point
Operating System	Software managing hardware + running applications	The manager of the building
Linux Kernel	Core layer between hardware and software	The engine — not the whole car
Linux Distribution	Kernel + bundled software = complete OS	RHEL (enterprise) or Ubuntu (cloud/startup)
Shell	Command line interpreter / user interface	$ = normal user; # = root
AlmaLinux / Rocky Linux	Free RHEL alternatives	Use for RHEL practice without a licence

PHASE 2 — Core Building Blocks: Filesystem, Permissions, Users & Groups

2.1 The Linux Filesystem Hierarchy

Linux organises all files in a single tree structure that starts at the root, written as / (forward slash). There are no drive letters like C: or D: — everything hangs off a single root. Think of it as a large company building: / is the lobby, and every floor and room branches off from there. Understanding this layout helps you find configuration files, log files, and application binaries without guessing.

Directory	What It Contains	Interview Tip
/	Root of the entire filesystem — the starting point of everything	Nothing is above /
/bin	Essential user binaries (commands): ls, cp, mv, cat, rm	Available to all users
/sbin	System administration binaries: fsck, mount, ifconfig	Typically only run by root
/etc	System-wide configuration files	Apache config, SSH config, user list — all here
/home	Home directories for regular users: /home/allen	Separate user data from OS data
/root	Home directory of the root account (NOT /home/root)	Root's home is /root not /home/root
/tmp	Temporary files — cleared at boot on most distros	Do not store anything important here
/var	Variable/changing data: log files, mail, spool	/var/log is where system logs live
/var/log	System and application log files	First place to look when troubleshooting
/usr	User programs and data: usr/bin, usr/lib, usr/sbin	Most installed application binaries go here
/opt	Optional / third-party software not bundled with OS	Google Chrome, custom apps: /opt/myapp
/boot	Files needed to boot: kernel (vmlinuz), initrd, GRUB	Do not delete anything from /boot
/dev	Device files: disks (/dev/sda), terminals (tty), null	Everything in Linux is a file — including hardware
/proc	Virtual filesystem for kernel/process information	Not real files — generated by the kernel on the fly
/mnt	Temporary mount points for filesystems	Mount extra drives here temporarily
/media	Auto-mount point for removable media (USB, CD)	Plug in a USB and it appears here
/srv	Data served by the system (web files, FTP files)	/srv/www for web data on some distros
/lib	Shared libraries needed by /bin and /sbin	Like DLL files in Windows

Applications not bundled with the OS are typically installed in either /opt or /usr/local. A common pattern: /opt/myapp/bin (binary), /opt/myapp/etc (config), /opt/myapp/log (logs). You will also see: /usr/local/bin (shared binary), /usr/local/etc/myapp.conf (config).

⚠ NOTE: /root is the home directory of the root account. /home/root does NOT exist. This is a very common confusion in interviews.

2.2 Linux File Permissions

Every file and directory in Linux has a set of permissions that controls who can read it, write to it, or execute it. This is one of the most important topics in Linux — and one of the most tested

in interviews. Think of permissions as a bouncer at a nightclub with three doors: one for the file's owner, one for the owner's group, and one for everyone else. Each door can be opened or locked independently.

2.2.1 Reading the Permission String

When you run ls -l, the first column shows the permission string. For example:

```
-rwxr-xr--  1  allen  devops  4096  Apr 1 10:00  myscript.sh
```

The permission string has 10 characters. Here is what each one means:

- `-` → File type: - = regular file, d = directory, l = symbolic link
- `rwx` → Owner permissions: r=read, w=write, x=execute
- `r-x` → Group permissions: r=read, -=no write, x=execute
- `r--` → Others permissions: r=read only, no write, no execute

2.2.2 Permission Types — Files vs Directories

Permission	On a File	On a Directory
r (read)	View file contents (cat, less, head)	List filenames inside the directory (ls)
w (write)	Modify file contents	Create, delete, or rename files inside the directory
x (execute)	Run the file as a program	Enter the directory (cd) and see file metadata (ls -l)

⚠ NOTE: Execute on a directory means you can cd into it and see file metadata like size and owner. Read on a directory means you can only see filenames. Without execute, you cannot even cd into the directory even if you know the filename inside it.

2.2.3 Numeric (Octal) Permissions

Each permission (r, w, x) has a numeric value: r=4, w=2, x=1. You add them together for each group (owner, group, others) to get a 3-digit octal number.

Octal	Symbolic	Meaning
7	rwx	Full permissions — read + write + execute (4+2+1)
6	rw-	Read and write only (4+2)
5	r-x	Read and execute only (4+1)
4	r--	Read only (4)
0	---	No permissions at all

755	rwxr-xr-x	Owner: full. Group and others: read+execute. Common for executables.
644	rw-r--r--	Owner: read+write. Group and others: read only. Common for files.
777	rwxrwxrwx	Everyone has full permissions. Dangerous — avoid unless necessary.
700	rwx------	Only the owner can access. Private scripts.
600	rw-------	Only the owner can read/write. Private files (SSH keys).

2.2.4 chmod — Changing Permissions

chmod stands for "change mode." You use it to change the permissions on a file or directory. There are two ways to use it: symbolic notation (letters) or octal notation (numbers).

```
# Symbolic notation
chmod u+x myscript.sh       # Add execute for the owner (user)
chmod g-w report.txt        # Remove write from the group
chmod o=r config.conf       # Set others to read-only exactly
chmod a+x deploy.sh         # Add execute for everyone (all)

# Octal notation
chmod 755 myscript.sh       # rwxr-xr-x
chmod 644 config.conf       # rw-r--r--
chmod 600 id_rsa            # rw------- (private key)
chmod -R 755 /opt/myapp     # Recursive: apply to all files inside
```

chmod → change mode — modifies the permission bits of a file or directory

u → user (owner of the file)

g → group (the group the file belongs to)

o → others (everyone who is not the owner and not in the group)

a → all (u + g + o combined)

+ → add the permission

- → remove the permission

= → set the permission exactly to this (remove all others)

-R → recursive — apply the change to all files inside the directory too

2.2.5 chown and chgrp — Changing Ownership

```
chown allen myscript.sh             # Change owner to allen
```

```
chown allen:devops myscript.sh      # Change owner to allen, group to devops
chown -R www-data:www-data /var/www/html   # Recursive ownership change
chgrp devops report.txt             # Change group only
```

`chown` → change owner — changes who owns the file

`allen:devops` → format is user:group — set both in one command

`-R` → recursive — apply to all files and subdirectories

`chgrp` → change group — changes only the group, not the owner

2.2.6 umask — Default Permission Mask

When a new file or directory is created, its permissions are not random — they are determined by the umask. The umask subtracts permissions from the system defaults. The system default for new files is 666 (rw-rw-rw-) and for new directories is 777 (rwxrwxrwx). The umask is then subtracted from these defaults.

```
umask           # Display current umask (shows as 4 digits, e.g. 0022)
umask 022       # Set umask to 022 (the most common setting)
umask -S        # Display umask in symbolic notation
```

umask	Default file permissions	Default directory permissions	Use case
022	644 (rw-r--r--)	755 (rwxr-xr-x)	Standard. Group/others cannot write.
002	664 (rw-rw-r--)	775 (rwxrwxr-x)	Group collaboration. Group can write.
077	600 (rw-------)	700 (rwx------)	Private. Only owner can access.
007	660 (rw-rw----)	770 (rwxrwx---)	Owner and group only. Others excluded.

⚠ NOTE: umask works opposite to chmod. chmod 7 means "give all permissions." umask 7 means "remove all permissions." Think of umask as what you take away from the default, not what you give.

2.3 Users and Groups

Linux is a multi-user operating system. Every process and every file is owned by a user. Users can belong to groups, and groups allow multiple users to share access to the same files and directories without giving access to everyone.

2.3.1 User Account Types

Account Type	UID Range	Purpose	Home Directory

Root (superuser)	0	Full unrestricted access to everything on the system	/root
System accounts	1–999	Run background services: www-data, nginx, mysql, nobody	Usually /var or /nonexistent
Regular users	1000+	Normal human users with limited privileges	/home/username

2.3.2 Key User and Group Files

```
/etc/passwd    # List of all user accounts
/etc/shadow    # Hashed passwords (readable only by root)
/etc/group     # Group definitions and membership
```

The /etc/passwd file has one line per user in this format:

```
username : x : UID : GID : GECOS/comment : home_directory : login_shell
```

`username` → the account name (e.g. allen)

`x` → password placeholder — actual hash is in /etc/shadow

`UID` → User ID — a unique number identifying the user (root = 0)

`GID` → Primary Group ID — the user's default group

`GECOS` → Optional comment/full name field

`home_directory` → path to the user's home directory (e.g. /home/allen)

`login_shell` → the shell the user gets on login (e.g. /bin/bash)

2.3.3 User Management Commands

```
# Create a new user
useradd -m -s /bin/bash -G devops allen

# Set/change password
passwd allen

# Modify an existing user
usermod -aG docker allen         # Add allen to docker group
usermod -s /bin/zsh allen        # Change shell

# Delete a user
userdel allen                    # Keep home directory
userdel -r allen                 # Also delete home directory

# Display user info
id allen                         # Shows UID, GID, and all groups
groups allen                     # Show groups allen belongs to
whoami                           # Show current username
```

```
who                                # Show who is logged in
last                               # Show login history
```

`useradd -m` → -m creates the home directory at /home/username

`useradd -s /bin/bash` → -s sets the login shell

`useradd -G devops` → -G adds user to supplementary group(s)

`usermod -aG docker` → -a means append (do not remove from existing groups); -G specifies groups

`userdel -r` → -r removes the home directory and mail spool too

`id` → shows UID (user ID), GID (primary group ID), and groups (all groups)

2.3.4 Group Management Commands

```
groupadd devops                    # Create a new group
groupmod -n infrastructure devops  # Rename group
groupdel devops                    # Delete a group
groups                             # List groups current user belongs to
groups allen                       # List groups allen belongs to
newgrp devops                      # Switch primary group in current session
```

2.4 Phase 2 Summary Table

Concept	Command / Tool	What It Does
Filesystem root	/	Starting point of all files and directories
View permissions	ls -l	Shows permission string, owner, group, size, date
Change permissions	chmod 755 / chmod u+x	Modify read/write/execute bits
Change owner	chown user:group	Change who owns the file
Default permissions	umask 022	Controls permissions on newly created files/dirs
Create user	useradd -m -s /bin/bash	Create new user account with home dir and shell
Set password	passwd username	Set or change a user's password
List groups	groups / id	Show group membership for a user
User account file	/etc/passwd	One line per user: name:x:UID:GID:comment:home:shell
Password hash file	/etc/shadow	Hashed passwords — root access only
Group file	/etc/group	Group names, GIDs, and members

PHASE 3 — Syntax and Configuration: The Command Line in Depth

3.1 Navigating the Filesystem

Navigation at the Linux command line uses a small set of commands that you will use dozens of times every day. Mastering these feels like muscle memory once you practice them.

```
pwd                     # Print working directory — where am I right now?
cd /var/log             # Change to absolute path
cd logs                 # Change to relative path (subdirectory of current location)
cd ..                   # Go up one level to parent directory
cd -                    # Go back to the previous directory (like browser back
button)
cd ~                    # Go to your home directory
cd                      # Same as cd ~ — goes home with no argument
```

pwd → print working directory — shows full path of where you are

cd → change directory

. → single dot = this directory (current directory)

.. → double dot = parent directory (one level up)

- → dash = previous directory (stored in $OLDPWD environment variable)

~ → tilde = shorthand for your home directory (/home/allen)

⚠ NOTE: cd - is extremely useful when you are switching back and forth between two directories. It acts exactly like the back button in a browser.

3.2 Listing Files — The ls Command

ls is the most commonly used Linux command. Understanding its output is fundamental to everything else.

```
ls                      # Basic listing
ls -l                   # Long format: permissions, links, owner, group, size, date,
name
ls -a                   # All files including hidden (dot files)
ls -la                  # Long format + hidden files (combine flags)
ls -lh                  # Long format with human-readable sizes (KB, MB, GB)
ls -lt                  # Long format sorted by modification time (newest first)
ls -latr                # Long + all + time-sorted + reversed (most recent at
bottom)
ls -F                   # Append type indicators: / for dir, @ for link, * for
executable
ls -R                   # Recursive listing of all subdirectories
ls -d /etc              # Show the directory itself, not its contents
ls --color              # Colorised output
```

Understanding a long listing line:

```
-rwxr-xr--  2  allen  devops  4096  Apr 01 10:00  deploy.sh
```

-rwxr-xr-- → Permission string: type + owner permissions + group permissions + others permissions

`2` → Number of hard links to this file

`allen` → Owner (the user who owns this file)

`devops` → Group (the group this file belongs to)

`4096` → Size in bytes

`Apr 01 10:00` → Last modification time

`deploy.sh` → The file or directory name

3.2.1 Symbolic Links

A symbolic link (symlink) is a pointer — it looks like a file or directory but just points to where the actual file lives. It appears in ls -l output with l as the first character, and with ls -F it ends with @. Symlinks are commonly used to create shortcuts to long paths or to point a generic name like "current" at a specific version directory.

```
ln -s /opt/myapp-2.4.7  /opt/myapp/current    # Create symlink "current" pointing
to version
ls -lF /opt/myapp/current                      # Shows: current@ -> /opt/myapp-
2.4.7
```

3.3 Creating and Managing Files and Directories

```
# Files
touch file.txt                    # Create empty file, or update timestamp if
exists
cp source.txt destination.txt     # Copy a file
cp -r /opt/myapp /opt/myapp-bak   # Copy directory recursively
cp -i source.txt dest.txt         # Interactive: prompt before overwriting
mv oldname.txt newname.txt        # Rename a file
mv file.txt /tmp/                 # Move a file to another directory
mv -i source.txt dest.txt         # Interactive: prompt if destination exists
rm file.txt                       # Delete a file (NO UNDO)
rm -r mydir                       # Delete directory and all contents recursively
rm -rf mydir                      # Force delete recursively (no prompts)

# Directories
mkdir newdir                      # Create a single directory
mkdir -p /opt/myapp/bin/scripts   # Create full path including parent directories
rmdir emptydir                    # Remove empty directory only
```

⚠ NOTE: rm -rf has no undo, no trash can, and no confirmation. Once you delete something, it is gone permanently. Always double-check your path before running rm -rf. A common safety habit: run ls against the path first, then run rm.

3.4 Viewing File Contents

```
cat /etc/hostname                  # Display entire file contents at once
head /var/log/syslog              # Display first 10 lines
head -20 /var/log/syslog          # Display first 20 lines
tail /var/log/syslog              # Display last 10 lines
tail -20 /var/log/syslog          # Display last 20 lines
tail -f /var/log/syslog           # Follow file in real time (watch new lines
arrive)
more /etc/nginx/nginx.conf        # Page through file (space=next page, q=quit)
less /etc/nginx/nginx.conf        # Like more but with more features (arrow keys,
search)
```

`cat` → concatenate — displays entire file at once, good for small files

`head` → shows the beginning of a file — useful for checking file format

`tail` → shows the end of a file — useful for checking recent entries

`tail -f` → follow mode — keeps watching the file and displays new lines as they are added. Essential for monitoring log files in real time

`less` → paginator with navigation: space=forward, b=back, /=search, q=quit. "Less is more" — it is an improvement on the more command

⚠ NOTE: Use tail -f to monitor logs while an application is running. For example, when troubleshooting a web server: tail -f /var/log/nginx/error.log

3.5 The Vi/Vim Editor

Vi (and its improved version Vim) is available on every Linux system. You will encounter it on servers where nano may not be installed. Vi has three modes:

Mode	How to Enter	What You Can Do
Command mode	Default when Vi opens; press Esc at any time	Navigate, search, delete, copy, paste
Insert mode	Press i (insert at cursor) or a (append after cursor)	Type text into the file
Line mode	Press : (colon) while in command mode	Save, quit, go to line number, search/replace

```
# Open a file
vi filename.txt

# Navigation in command mode
h  j  k  l          # Left, down, up, right (like arrow keys)
w                    # Jump forward one word
b                    # Jump back one word
^                    # Go to beginning of line
$                    # Go to end of line
```

```
# Entering insert mode
i                       # Insert at cursor
I                       # Insert at beginning of line
a                       # Append after cursor
A                       # Append at end of line

# Saving and quitting (line mode, press : first)
:w                      # Save (write) the file
:w!                     # Force save
:q                      # Quit (only works if no unsaved changes)
:q!                     # Force quit without saving
:wq                     # Save and quit
:wq! or :x              # Force save and quit

# Useful command mode operations
x                       # Delete character under cursor
dd                      # Delete (cut) entire current line
yy                      # Yank (copy) entire current line
p                       # Paste below current line
u                       # Undo
5dd                     # Delete 5 lines (repeat any command with a number)

# Line mode navigation and search
:15                     # Go to line 15
:$                      # Go to last line
:set nu                 # Show line numbers
:set nonu               # Hide line numbers
/searchterm             # Search forward
n                       # Go to next search result
```

⚠ NOTE: If you get stuck in Vi and cannot exit, press Esc first (to get to command mode), then type :q! and press Enter. This force-quits without saving. Memorise this — you will need it.

3.6 Wildcards and Globbing

Wildcards are special characters that match patterns in filenames. The shell expands them into a list of matching files before passing them to the command. This is called globbing.

Wildcard	Matches	Example	What It Matches
*	Zero or more characters (anything)	ls *.txt	All files ending in .txt
?	Exactly one character	ls file?.txt	file1.txt, fileA.txt — not file10.txt
[abc]	Any one of the listed characters	ls file[123].txt	file1.txt, file2.txt, file3.txt
[a-z]	Any character in the range	ls [a-d]*	All files starting with a, b, c, or d
[!abc]	NOT any of the listed characters	ls [!aeiou]*	Files not starting with a vowel
[[:alpha:]]	Named class: any letter	ls [[:upper:]]*	Files starting with

			uppercase letter
[[:digit:]]	Named class: any digit 0-9	ls report[[:digit:]]	report0 through report9

```
ls *.log                          # All files ending in .log
ls app-?.conf                     # app-1.conf, app-a.conf — one char after dash
rm *.tmp                          # Delete all .tmp files (be careful!)
cp /etc/*.conf /tmp/              # Copy all .conf files to /tmp
ls [Rr]eport*                     # Files starting with Report or report
ls ca[nt]*                        # can, cat, candy, catch
```

3.7 I/O Redirection

Every command in Linux deals with three standard data streams: stdin (input), stdout (output), and stderr (error messages). Redirection allows you to change where these streams go — to or from files, or between commands.

Operator	Name	What It Does	Example
>	Redirect stdout	Send output to a file (OVERWRITES)	ls -la > filelist.txt
>>	Append stdout	Append output to a file (does not overwrite)	echo "log entry" >> app.log
<	Redirect stdin	Use a file as input to a command	mail allen < message.txt
2>	Redirect stderr	Send error messages to a file	find / -name "*.conf" 2> /dev/null
2>&1	Merge stderr into stdout	Combine errors with normal output	find / -name "*.conf" > results.txt 2>&1
&>	Redirect both	Send both stdout and stderr to same file	command &> output.txt

```
# Redirect stdout to a file (overwrite)
ls -la /etc > /tmp/etc-listing.txt

# Append to an existing file
echo "$(date): job complete" >> /var/log/myjob.log

# Suppress error messages (send to /dev/null — the Linux "black hole")
find / -name "nginx.conf" 2> /dev/null

# Capture both output and errors
apt install nginx > /tmp/install.log 2>&1

# Use a file as input
sort < unsorted_names.txt
```

`/dev/null` → the "black hole" device — anything written here is discarded silently

`2>&1` → redirect file descriptor 2 (stderr) to wherever file descriptor 1 (stdout) is going

⚠ **NOTE: > overwrites the file completely. >> appends to it. A common mistake is using > when you meant >> and losing all your existing log data.**

3.8 Pipes and Text Processing Commands

The pipe character | takes the stdout of one command and sends it as the stdin of the next command. This is the Unix philosophy in action: build small tools that do one thing well, then chain them together to do something powerful.

```
# Basic pipe
cat /etc/passwd | less                          # Page through the passwd file

# Find all Bob entries in passwd, show username and full name, sort alphabetically
grep "bob" /etc/passwd | cut -d: -f1,5 | sort | tr ":" " " | column -t

# Count lines matching a pattern
grep "ERROR" /var/log/app.log | wc -l

# Show the 5 largest files in current directory
ls -lS | head -6

# Extract readable strings from a binary, search for a name
strings giant-steps.mp3 | grep -i "john"
```

Key text processing commands used in pipelines:

Command	Purpose	Key Options
grep pattern file	Search for pattern in file or stdin	-i case-insensitive; -v invert; -n line numbers; -c count; -r recursive
cut -d: -f1,5 file	Extract specific columns from delimited text	-d delimiter character; -f field numbers (comma-separated)
sort file	Sort lines alphabetically or numerically	-r reverse; -k field; -u unique (remove duplicates)
tr ":" " "	Translate (replace) characters	tr old new; tr -d "char" to delete characters
wc file	Count lines, words, characters	-l lines only; -w words only; -c characters only
uniq	Remove adjacent duplicate lines (use with sort)	-c count occurrences
column -t	Format input as aligned table	-t auto-detect columns
head -N	Show first N lines	-N e.g. head -20
tail -N	Show last N lines / follow file	-N or -f for follow mode
strings file	Extract human-readable strings from binary files	useful for searching audio/binary for metadata

3.9 Searching for Files and Directories

3.9.1 find — Real-time File Search

find searches the filesystem in real time, evaluating every file and directory against your criteria. It is powerful, flexible, and always up to date.

```
# Find by name (case-sensitive)
find /etc -name "nginx.conf"

# Find by name (case-insensitive)
find /etc -iname "nginx.conf"

# Find by type (f=file, d=directory, l=symlink)
find /var -type f
find /opt -type d

# Find files modified in last 7 days
find /var/log -mtime -7

# Find files larger than 100MB
find / -size +100M

# Find all setuid files (security audit)
find / -perm /4000 -ls

# Find and execute a command on results
find /tmp -name "*.tmp" -exec rm {} \;

# Suppress permission denied errors
find / -name "*.conf" 2>/dev/null
```

`find / -name "file"` → search the entire filesystem for files named "file"

`-mtime -7` → modified time less than 7 days ago (minus = within N days)

`-mtime +30` → modified more than 30 days ago (plus = older than N days)

`-size +100M` → larger than 100 megabytes

`-perm /4000` → forward slash means match any of the bits set — here the setuid bit

`-exec rm {} \;` → {} is replaced by the found file; \; terminates the -exec expression

`-ls` → display found files in long listing format (like ls -l)

3.9.2 locate — Fast Index-Based Search

locate searches a pre-built database of filenames rather than scanning the filesystem. It is much faster than find but has one major limitation: the database is only updated once per day, so newly created files will not appear yet.

```
locate nginx.conf                # Fast search for nginx.conf
```

```
locate -i nginx                    # Case-insensitive search
updatedb                           # Manually rebuild the database (run as root)
```

⚠ NOTE: If you create a file and immediately search for it with locate, it will not appear. Use find instead for newly created files. locate is best for finding system files and configuration files that have existed for a while.

	find	locate
Speed	Slower — reads filesystem in real time	Faster — searches pre-built database
Freshness	Always current (real time)	May be up to 24 hours out of date
New files	Finds them immediately	Does not find recently created files
Best for	Advanced searches, new files, permission searches	Quick lookups of well-known files

3.10 Environment Variables

Environment variables are named storage locations that store values affecting how programs behave. Think of them as global settings that applications can read. They are case-sensitive and by convention written in UPPERCASE.

```
# View all environment variables
printenv

# View a specific variable
printenv PATH
echo $PATH

# Create/update an environment variable (current session only)
export EDITOR="vim"
export TZ="Asia/Kolkata"

# Remove an environment variable
unset TZ

# Make it persist across logins — add to ~/.bash_profile
echo 'export EDITOR="vim"' >> ~/.bash_profile
```

Variable	What It Controls	Example Value
PATH	Colon-separated list of directories the shell searches for commands	/usr/local/bin:/usr/bin:/bin
HOME	Your home directory path	/home/allen
HOSTNAME	The name of the system	azure-vm-01
EDITOR	Your preferred text editor for commands that open editors	vim
TZ	Timezone override for date/time commands	Asia/Kolkata
HISTSIZE	Number of commands to store in shell history	1000
PS1	Controls the appearance of your	\u@\h:\w\$

	shell prompt	
OLDPWD	The previous directory (used by cd -)	/var/log

⚠ NOTE: Variables set with export only last for the current session. To persist them across logins, add the export command to ~/.bash_profile or ~/.bashrc. The shell reads these files when you log in.

3.11 Customising the Shell Prompt (PS1)

The PS1 environment variable controls what your shell prompt looks like. Customising it helps you always know your username, hostname, and current directory at a glance.

```
# View current PS1
echo $PS1

# Set a custom prompt: user@host:directory$
PS1="\u@\h:\w\$ "

# Multi-line prompt: time on first line, then host and directory
PS1="\t\n\u@\h:\w\$ "

# Persist in ~/.bash_profile
echo 'PS1="\u@\h:\w\$ "' >> ~/.bash_profile
```

Format Code	What It Displays	Example Output
\u	Current username	allen
\h	Short hostname (up to first dot)	azure-vm-01
\H	Full hostname (FQDN)	azure-vm-01.eastus.cloudapp.azure.com
\w	Full current working directory path	/var/log/nginx
\W	Basename (last part) of current directory	nginx
\d	Date in Weekday Month Day format	Thu Apr 01
\t	Current time in 24-hour format	14:32:55
\T	Current time in 12-hour format	02:32:55
\@	Current time 12-hour with AM/PM	02:32 PM
\n	Newline — creates a multi-line prompt	(new line)
\$	$ for normal user, # for root	$ or #

3.12 Shell Aliases

An alias is a shortcut — a custom name that expands into a longer command. Think of it as a text expander for the terminal. Once you define an alias, you can type the short name and the shell automatically replaces it with the full command.

```
# List all defined aliases
alias

# Create an alias
alias ll="ls -la"
alias grep="grep --color=auto"
alias cls="clear"              # Windows-style clear screen
alias ..="cd .."               # Go up one level
alias update="sudo apt update && sudo apt upgrade -y"

# Remove a specific alias
unalias ll

# Remove all aliases
unalias -a

# Persist aliases — add to ~/.bash_profile or ~/.bashrc
echo 'alias ll="ls -la"' >> ~/.bash_profile
```

⚠ NOTE: Aliases only exist in the current session unless saved to ~/.bash_profile. If you rely heavily on aliases and SSH into a system without your config, you will feel lost. Keep alias usage minimal for commands you must know the underlying syntax for.

3.13 Shell History

Bash records every command you type into your shell history. This saves enormous amounts of time — instead of retyping a complex command, you can recall and re-run it.

```
history                          # Show all history with line numbers
history 20                       # Show last 20 commands

# Re-run commands by history number
!42                              # Re-run command number 42
!!                               # Re-run the previous command
!ssh                             # Re-run the most recent command starting with
"ssh"

# Pull out arguments from the previous command
!:1                              # First argument of previous command
!:2                              # Second argument
!^                               # Same as !:1 — first argument
!$                               # Last argument of previous command
!*                               # All arguments of previous command

# Correct a typo: re-run grep with same arguments as failed grpe command
grep !*
```

```
# Create a user and run subsequent commands on same username
useradd jimmy
passwd !$                        # !$ = "jimmy" — the last arg of previous
command
chage -M 60 !$                   # Set password expiry for jimmy

# Search history interactively
Ctrl + R                         # Start reverse incremental search, type to
search
```

`history` → displays command history with numbers — HISTSIZE variable controls how many are kept (default 500)

`!!` → bang bang — re-runs the entire previous command. Useful: forgot to type sudo? Do: sudo !!

`!$` → bang dollar — last argument of previous command. Saves retyping long paths/names

`!*` → bang star — ALL arguments of previous command (not the command itself)

`Ctrl+R` → opens reverse search — type partial command, it finds the most recent matching one

⚠ NOTE: sudo !! is a very common pattern. You run a command, get a "Permission denied" error, then run sudo !! to re-run it with root privileges.

3.14 Getting Help

```
man ls                           # Manual page for ls command
man 5 /etc/passwd                # Man page section 5 (file formats)
man -k keyword                   # Search all man pages by keyword (like apropos)
apropos network                  # Search man pages for "network"
ls --help                        # Quick inline help
whatis ls                        # One-line description of a command
info ls                          # GNU info documentation (more detailed than
man)
which ls                         # Full path to the ls command executable
whereis ls                       # Find binary, source, and man page locations
```

Man page sections: 1=user commands, 2=system calls, 3=library functions, 4=special files, 5=file formats, 6=games, 7=misc, 8=system admin commands. When a name appears in multiple sections, specify the section: man 5 passwd (the file format) vs man passwd (the command).

3.15 Phase 3 Summary Table

Topic	Key Commands	Remember
Navigate	pwd, cd, cd .., cd -, cd ~	. = current dir; .. = parent; ~ = home; - = previous
List files	ls -la, ls -latr, ls -F	-latr = long+all+time+reverse (most recent at bottom)

Create/delete	touch, mkdir -p, rm -rf, cp -r, mv	rm -rf has no undo — always verify before running
View files	cat, head, tail, tail -f, less	tail -f is for real-time log monitoring
Edit files	vi / vim, nano	Esc then :q! to force-quit vi without saving
Wildcards	*, ?, [abc], [a-z], [!abc]	* = anything; ? = one char; [ ] = character class
Redirection	>, >>, <, 2>, 2>&1	> overwrites; >> appends; 2>/dev/null discards errors
Pipes	| grep, | cut, | sort, | wc	Chains stdout of one command into stdin of next
Find files	find / -name, locate	find = real time; locate = fast but may be 24h old
Environment	printenv, export, unset	Persist in ~/.bash_profile
Prompt	PS1=	\u=user \h=host \w=directory \$=$ or #
Aliases	alias, unalias	Persist in ~/.bash_profile
History	history, !!, !$, Ctrl+R	sudo !! is the most common use of !!

PHASE 4 — Workflow: Processes, Jobs, Packages, Users & Network

4.1 Process and Job Management

Every program that is running on a Linux system is a process. Each process has a unique PID (Process ID). Processes can run in the foreground (you see their output and cannot use the shell until they finish) or the background (they run silently and you get your shell prompt back immediately).

4.1.1 Viewing Processes

```
ps                                  # Processes in your current session
ps -e                               # All processes on the system
ps -ef                              # All processes, full format (UID, PID, PPID,
command)
ps -eH                              # All processes in tree format (shows parent-
child)
ps --forest                         # Same as -H but more visual
ps -u allen                         # All processes owned by user allen
ps -p 1234                          # Details about process with PID 1234

pstree                              # Visual tree of all processes
top                                 # Interactive real-time process viewer
htop                                # Enhanced version of top (may need to install)
```

`ps -ef` → e = all processes; f = full format showing UID, PID, PPID, time, command

`PPID` → Parent Process ID — the PID of the process that created this process

`top` → live-updating view; press q to quit; k to kill a process by PID; h for help

4.1.2 Foreground and Background Jobs

```
# Run a command in the background
sleep 300 &

# List background/suspended jobs
jobs

# Bring job 1 back to foreground
fg %1

# Send a foreground job to background
# First suspend it with Ctrl+Z, then:
bg %1

# Key shortcuts
Ctrl + C      # Terminate the foreground process (sends SIGINT)
Ctrl + Z      # Suspend (pause) the foreground process (sends SIGTSTP)
```

`&` → ampersand at end of command = run in background; shell immediately returns to prompt

`jobs` → lists all background and suspended jobs with their job numbers [1], [2]...

`fg %1` → bring job number 1 to the foreground

`bg %1` → resume suspended job 1 in the background

`Ctrl+Z` → suspend (pause) the running foreground process — it keeps the process alive but stopped

4.1.3 Killing Processes

```
kill 1234                        # Send SIGTERM (signal 15) — polite request to
stop
kill -9 1234                     # Send SIGKILL — force kill, cannot be caught or
ignored
kill -15 1234                    # Explicit SIGTERM (same as kill 1234)
kill -l                          # List all available signals
killall nginx                    # Kill all processes named nginx
```

`kill 1234` → sends SIGTERM (15) to PID 1234 — asks the process to clean up and exit gracefully

`kill -9 1234` → SIGKILL — the kernel kills the process immediately, no cleanup possible. Use as last resort.

`killall nginx` → kills all processes named nginx — useful when you have multiple worker processes

⚠ **NOTE: Always try kill (SIGTERM) first. Only use kill -9 if the process is frozen and not responding to SIGTERM. A process killed with -9 cannot clean up its resources, close files, or write final state.**

4.2 Scheduling with Cron

Cron is the Linux job scheduler. It allows you to run commands automatically at specified times. Think of it as a personal assistant that runs tasks for you while you sleep — it works from a schedule you define in a crontab file.

```
crontab -l                          # List your current cron jobs
crontab -e                          # Edit your cron jobs (opens in $EDITOR)
crontab -r                          # Remove ALL your cron jobs (dangerous!)
crontab filename                    # Install crontab from a file
```

Each crontab entry has this format:

```
# minute  hour  day_of_month  month  day_of_week  command
#  (0-59) (0-23)   (1-31)    (1-12)   (0-6)       /path/to/command

# Examples:
0 7 * * 1    /opt/sales/bin/weekly-report.sh   # Every Monday at 7:00 AM
0 0 * * *    /opt/scripts/nightly-backup.sh    # Every day at midnight
*/5 * * * *  /opt/scripts/healthcheck.sh       # Every 5 minutes
0 */2 * * *  /opt/scripts/sync.sh              # Every 2 hours
30 8 1 * *   /opt/scripts/monthly.sh           # 1st of every month at 8:30 AM
0 9 * * 1-5  /opt/scripts/workday.sh           # Weekdays at 9:00 AM
```

`*` → asterisk = any value for this field (matches everything)

`*/5` → every 5 units — */5 in minutes means "every 5 minutes"

`1-5` → range — days 1 through 5 in day_of_week means Monday through Friday

`0,30` → list — run at minute 0 and minute 30 of every hour

`1` → in day_of_week: 0=Sunday, 1=Monday, 2=Tuesday ... 6=Saturday

Cron shortcut keywords (not available on all implementations — check with man cron):

Shortcut	Equivalent	When It Runs
@yearly or @annually	0 0 1 1 *	Once a year at midnight on January 1st
@monthly	0 0 1 * *	Once a month at midnight on the 1st
@weekly	0 0 * * 0	Once a week at midnight on Sunday
@daily or @midnight	0 0 * * *	Every day at midnight
@hourly	0 * * * *	At the start of every hour

⚠ **NOTE: Cron output (stdout and stderr) is emailed to the user by default. To suppress this, redirect output: 0 7 * * 1 /script.sh > /dev/null 2>&1**

4.3 Switching Users and sudo

In real-world DevOps work you will constantly need to run commands with elevated privileges, or run processes as service accounts (like www-data for web servers, or oracle for a database).

4.3.1 su — Switch User

```
su                                # Switch to root (asks for root password; keeps
your environment)
su -                              # Switch to root WITH root's full environment
su - allen                        # Switch to user allen with allen's full
environment
su -c "systemctl restart nginx" root   # Run single command as root, then return

# After switching users, verify who you are:
whoami
```

`su` → substitute user — switches to another account

`su -` → the hyphen/dash creates a login shell — gives you the full environment of the target user (their PATH, their HOME, their shell). Always use su - unless you specifically need to keep your own environment.

⚠ **NOTE: su - (with the dash) vs su (without the dash) is a common interview question. Without the dash: you switch user but keep YOUR environment variables. With the dash: you get a complete login environment as the target user — their PATH, HOME, and shell config are all loaded.**

4.3.2 sudo — Super User Do

sudo allows you to run a command as root (or another user) using YOUR OWN password. This is more secure than sharing the root password and creates an audit trail in /var/log/auth.log.

```
sudo apt install nginx             # Run apt as root using your password
sudo -u www-data php /opt/app.php  # Run as user www-data
sudo -l                            # List what sudo commands you are allowed to
run
sudo su -                          # Get a root shell via sudo
sudo -s                            # Start a root shell (uses current environment)

# Edit the sudoers file safely
visudo                             # ALWAYS use visudo — it validates syntax
before saving
```

The /etc/sudoers file format:

```
# Format: WHO  WHERE=(AS_WHOM)  WHAT
allen  ALL=(ALL)  ALL                      # allen can run any command as any
user
allen  ALL=(ALL)  NOPASSWD: ALL            # Same but no password required
allen  ALL=(root) /usr/bin/systemctl       # allen can only run systemctl as root
%devops  ALL=(ALL)  ALL                    # All members of devops group get full
sudo
```

`allen` → the user or %groupname who gets the privilege

`ALL` → first ALL = on any host; (ALL) = as any user; last ALL = any command

`NOPASSWD:` → no password prompt when running sudo — commonly used for automation scripts

`visudo` → opens sudoers in the editor and validates syntax before saving — prevents locking yourself out

⚠ NOTE: Never edit /etc/sudoers directly with vi or nano. Always use visudo. A syntax error in /etc/sudoers can prevent anyone from using sudo, potentially locking you out of the system.

4.4 Package Management

Linux applications are distributed as packages. A package contains the application binary, configuration files, documentation, and metadata about dependencies (other packages it needs). Package managers handle installation, removal, and updates automatically including dependencies.

4.4.1 RPM-Based Systems (RHEL, AlmaLinux, Rocky Linux, Fedora)

```
# dnf — high-level package manager (recommended)
dnf search nginx                    # Search for a package
dnf info nginx                      # Show package details
dnf install nginx                   # Install nginx
dnf remove nginx                    # Remove nginx
dnf update                          # Update all packages
dnf update nginx                    # Update only nginx
dnf list installed                  # List all installed packages
dnf autoremove                      # Remove unused dependencies

# rpm — low-level package tool (for .rpm files)
rpm -ivh package.rpm                # Install: i=install, v=verbose, h=show hash
progress
rpm -e nginx                        # Erase (remove) a package
rpm -qa                             # Query all installed packages
rpm -qf /usr/bin/nginx              # Which package owns this file
rpm -ql nginx                       # List all files in an installed package
```

`dnf` → Dandified YUM — the modern package manager for RPM-based systems. Handles dependencies automatically.

`yum` → the predecessor to dnf, still works on many systems. Same syntax as dnf mostly.

rpm → low-level tool — does NOT resolve dependencies automatically. Use dnf/yum for installs when possible.

rpm -qf /path → which package owns this file — very useful for troubleshooting

4.4.2 Debian-Based Systems (Ubuntu, Debian)

```
# apt — high-level package manager (recommended)
apt update                          # Refresh the package index from repositories
apt upgrade                         # Upgrade all installed packages
apt upgrade -y                      # Upgrade without confirmation prompts
apt install nginx                   # Install nginx
apt remove nginx                    # Remove nginx (keep config files)
apt purge nginx                     # Remove nginx AND its config files
apt search nginx                    # Search for packages
apt show nginx                      # Show package details
apt list --installed                # List all installed packages
apt autoremove                      # Remove packages no longer needed

# dpkg — low-level package tool (for .deb files)
dpkg -i package.deb                 # Install a .deb package
dpkg -r nginx                       # Remove a package
dpkg -l                             # List all installed packages
dpkg -L nginx                       # List all files belonging to nginx package
dpkg -S /usr/sbin/nginx             # Which package owns this file
```

apt update → DOES NOT update packages — it only refreshes the local list of available packages. Always run this before apt install.

apt upgrade → ACTUALLY upgrades packages to their latest versions

apt autoremove → removes packages that were installed as dependencies but are no longer needed

dpkg → like rpm — low-level, no dependency resolution. Use apt when possible.

⚠ NOTE: Always run apt update before apt install. If you forget, you may install an outdated version or get errors about packages not being found.

4.5 SSH — Connecting to Linux Remotely

SSH (Secure Shell) is how DevOps engineers connect to Linux servers. It provides an encrypted command-line connection over the network. In an Azure environment, you will use SSH to connect to your VMs constantly.

```
# Connect to a remote server
ssh allen@10.0.0.4
ssh allen@azure-vm-01.eastus.cloudapp.azure.com

# Connect on a non-standard port
ssh -p 2222 allen@10.0.0.4
```

```
# SSH key-based authentication (more secure — no password needed)
ssh-keygen -t rsa -b 4096           # Generate key pair
ssh-copy-id allen@10.0.0.4          # Copy public key to remote server
ssh allen@10.0.0.4                  # Now connects without password

# Key files
~/.ssh/id_rsa                       # Private key (NEVER share this)
~/.ssh/id_rsa.pub                   # Public key (safe to share/copy)
~/.ssh/authorized_keys              # Keys allowed to connect to this account
~/.ssh/known_hosts                  # Fingerprints of servers you've connected to
~/.ssh/config                       # Client-side SSH configuration
```

`ssh-keygen` → generates a public/private key pair — like a padlock (public) and key (private)

`ssh-copy-id` → copies your public key to the remote server's ~/.ssh/authorized_keys file

`authorized_keys` → the server checks this file — if your public key is listed, you can connect without a password

⚠ NOTE: In Azure, when creating a Linux VM, you provide your public key and Azure puts it in the VM's authorized_keys. You then connect using your private key. The az aks get-credentials command and similar tools follow the same principle.

4.6 Transferring Files Over the Network

Tool	Protocol	Use Case	Security
scp	SSH	Copy specific files when you know source and destination paths	Encrypted
sftp	SSH	Interactive session — browse remote, put/get files	Encrypted
ftp	FTP	Legacy — avoid unless forced to use it	INSECURE — plaintext credentials
FileZilla / Cyberduck	SFTP/SCP	Graphical clients for Windows/Mac	Encrypted (via SFTP)
WinSCP	SFTP/SCP	Windows-only graphical client	Encrypted
pscp.exe	SCP	PuTTY's command-line SCP for Windows	Encrypted

```
# scp — Secure Copy
scp file.txt allen@10.0.0.4:/home/allen/          # Local to remote
scp allen@10.0.0.4:/home/allen/file.txt /tmp/     # Remote to local
scp -r /opt/myapp allen@10.0.0.4:/opt/            # Copy directory recursively

# sftp — interactive session
sftp allen@10.0.0.4
   ls                   # List remote files
   lls                  # List LOCAL files (l prefix = local)
```

```
pwd                     # Show remote working directory
lpwd                    # Show LOCAL working directory
put file.txt            # Upload file to remote
get file.txt            # Download file from remote
cd /var/log             # Change remote directory
lcd /tmp                # Change LOCAL directory
exit                    # Disconnect
```

⚠ NOTE: FTP is insecure — your username, password, and file contents are all sent in plaintext. Anyone on the network can intercept them. Always use SCP or SFTP instead. If you see FTP in an interview question about security, the answer is always: prefer SFTP/SCP.

4.7 Phase 4 Summary Table

Topic	Key Commands	Key Insight
View processes	ps -ef, top, pstree	Every running program is a process with a PID
Kill processes	kill PID, kill -9 PID	Try SIGTERM first; -9 (SIGKILL) is last resort
Background jobs	command &, jobs, fg, bg	Ctrl+Z suspends; & runs in background from start
Cron scheduling	crontab -e/-l/-r	Format: min hour dom month dow command
Switch user	su -, sudo command	su - gives full login environment; sudo uses your password
sudoers	visudo	Always use visudo — validates syntax before saving
Packages (Ubuntu)	apt update, apt install	update = refresh index; upgrade = actually update
Packages (RHEL)	dnf search, dnf install	dnf replaces yum; rpm is low-level
SSH	ssh user@host, ssh-keygen	Public key in authorized_keys = no password needed
File transfer	scp, sftp	Both encrypted; prefer over insecure FTP

PHASE 5 — Real World Structure: Boot, Logging, Disks, and Networking

5.1 The Linux Boot Process

Understanding the boot process is essential for troubleshooting servers that will not start, recovering from disasters, and administering systems at a deep level. The boot process has four main stages.

Stage	Component	What Happens

1	BIOS / UEFI	Power on → POST (Power-On Self-Test checks CPU, RAM, storage) → find bootable device → hand off to boot loader
2	Boot Loader (GRUB)	Displays boot menu → loads the selected kernel and initrd into RAM → passes control to kernel
3	Kernel + initrd	initrd (temporary filesystem) loads hardware drivers → mounts the real root filesystem → starts first process
4	Init / systemd	PID 1 starts → loads targets/runlevels → starts all services → presents login prompt

5.1.1 BIOS and UEFI

BIOS (Basic Input Output System) is the firmware that runs first when a computer powers on. It performs a POST (Power-On Self-Test) to check basic hardware — CPU, memory, storage. If the POST succeeds, BIOS searches for a bootable device in the configured order (e.g. USB first, then hard disk). UEFI is the modern replacement for BIOS and supports GPT disk partitioning and larger drives.

5.1.2 Boot Loaders — GRUB and LILO

Once BIOS hands off to a bootable device, the boot loader runs. On modern Linux systems, this is GRUB (Grand Unified Bootloader). LILO (Linux Loader) is an older alternative you may see on legacy systems. GRUB's primary job is to start the operating system — it reads its configuration, presents a menu (if multiple kernels are installed), and loads the selected kernel and initrd into memory. You can also use GRUB to pass boot arguments to the kernel (e.g. single user mode recovery: add the letter s to the kernel line).

The /boot directory holds all files needed for booting:

```
ls /boot
# vmlinuz-5.15.0-generic      ← Compressed Linux kernel (z = compressed)
# initrd.img-5.15.0-generic   ← Initial RAM disk
# grub/                        ← GRUB configuration directory
```

`vmlinuz` → the Linux kernel image — vmlinux = uncompressed; vmlinuz = compressed (z = compressed)

`initrd.img` → initial RAM disk — a temporary filesystem loaded into RAM to help mount the real root filesystem

5.1.3 initrd — Initial RAM Disk

The initrd is a temporary, minimal filesystem that is loaded into RAM during boot. Its job is to load the kernel modules (drivers) needed to mount the real root filesystem. This is critical when the root filesystem lives on LVM volumes, RAID arrays, or encrypted disks — the kernel needs those drivers before it can mount the disk. Once initrd mounts the real root filesystem, it hands off control and disappears.

5.1.4 Runlevels vs. systemd Targets

After the kernel starts, it launches the first process. Traditionally this was init (PID 1) which used runlevels to determine what services to start. Modern Linux uses systemd as the replacement for init, and systemd uses "targets" instead of runlevels.

Runlevel	systemd Target	Description
0	poweroff.target	System halt / power off
1	rescue.target	Single-user mode — only root, minimal services (for recovery)
2-4	multi-user.target	Multi-user text mode — servers typically boot to this
5	graphical.target	Multi-user with graphical interface (desktop)
6	reboot.target	System reboot

```
# View current target
systemctl get-default

# Change default target
systemctl set-default multi-user.target   # Server — no GUI
systemctl set-default graphical.target    # Desktop with GUI

# Switch target immediately (without reboot)
systemctl isolate multi-user.target

# Reboot / power off
reboot                                    # Reboot immediately
poweroff                                  # Power off immediately
shutdown -r now                           # Reboot now
shutdown -r +5 "Rebooting in 5 minutes"  # Reboot in 5 min with message
shutdown -h 22:30                         # Halt at 10:30 PM

# Traditional init commands (older systems)
telinit 6                                 # Reboot (change to runlevel 6)
telinit 0                                 # Power off
```

5.1.5 Viewing Boot Messages

```
dmesg                                     # View kernel ring buffer messages
dmesg | less                              # Page through kernel messages
```

```
dmesg | grep -i error                    # Filter for errors in boot messages
cat /var/log/dmesg                       # Same messages stored on disk
```

⚠ NOTE: The kernel ring buffer is a fixed-size buffer — when full, old messages are discarded as new ones arrive. Use dmesg immediately after a reboot to see boot messages before they scroll off.

5.2 System Logging — rsyslog and logrotate

Linux systems generate a constant stream of log messages — from the kernel, from services, from authentication events. The syslog standard defines how these messages are categorised and stored.

5.2.1 The syslog Standard

Every syslog message has two labels: a facility (what generated the message) and a severity (how important it is). These two together determine where the message gets written.

Facility	Source of the Message
kern	Linux kernel
auth / authpriv	Authentication events (login attempts, sudo usage)
cron	Cron job scheduler
daemon	System daemons (background services)
mail	Mail subsystem
user	User-generated messages (default when not specified)
local0–local7	Custom facilities for application use

Severity (highest to lowest)	Meaning
emerg (0)	System is unusable — immediate action required
alert (1)	Action must be taken immediately
crit (2)	Critical condition (hardware failure)
err (3)	Error condition
warning (4)	Warning — potential problem
notice (5)	Normal but significant condition
info (6)	Informational message
debug (7)	Debug-level messages — very verbose

5.2.2 rsyslog — The Syslog Daemon

rsyslog is the most common syslog implementation. Its configuration file is /etc/rsyslog.conf, and per-service configurations go in /etc/rsyslog.d/. Rules are in the format: FACILITY.SEVERITY ACTION.

Log File	Distribution	What Gets Logged There
/var/log/messages	RHEL / AlmaLinux	General system messages (most things except auth, cron, mail)
/var/log/syslog	Ubuntu / Debian	Same as /var/log/messages but on Debian-based systems
/var/log/auth.log	Ubuntu / Debian	Authentication: SSH logins, sudo usage, failed passwords
/var/log/secure	RHEL / AlmaLinux	Same as auth.log on RHEL systems
/var/log/dmesg	All	Kernel ring buffer — hardware detection, driver messages
/var/log/cron	All	Cron job execution records
/var/log/maillog	RHEL	Mail subsystem messages

```
# Generate a test syslog message
logger "Test message from allen"
logger -p auth.info "SSH key test"
logger -p mail.info -t "myapp" "Application started"
```

`logger` → generates a syslog message from the command line — useful for testing logging config and for logging from shell scripts

`-p facility.severity` → specify the facility and severity of the message

`-t tag` → add a tag (application name) to identify the message source

5.2.3 logrotate — Automated Log Management

Log files grow continuously. Without management, they would fill up the disk and kill the server. logrotate automatically rotates (renames/compresses) log files on a schedule, keeping only a certain number of old versions.

```
# Main config file
cat /etc/logrotate.conf

# Per-application configs
ls /etc/logrotate.d/

# Force a rotation (test your config)
logrotate -fv /etc/logrotate.conf
```

Example logrotate configuration for an application:

```
/var/log/myapp/*.log {
    weekly              # Rotate every week
    rotate 4            # Keep 4 weeks of logs before deleting
    compress            # Compress old log files with gzip
    missingok           # Do not error if log file is missing
    notifempty          # Do not rotate if log file is empty
    create 0644 www-data www-data   # Create fresh log with these permissions
    postrotate          # Commands to run after rotation
```

```
        systemctl reload myapp
    endscript
}
```

weekly → rotate once a week

rotate 4 → keep 4 old versions; the 5th rotation deletes the oldest

compress → old rotated logs are compressed to .gz files to save space

missingok → no error if the log file does not exist — prevents failures during first run

notifempty → do not create an empty rotated file if the log has nothing in it

postrotate...endscript → run these commands after rotation — usually to signal the application to reopen its log file

logrotate -fv → -f forces rotation even if it is not due yet; -v verbose output for testing

5.3 Disk Management

5.3.1 Partitions and Partition Tables

A disk can be divided into partitions — separate logical sections each used for a different purpose. For example: one partition for the OS, one for /home (user data), one for /var (logs), one for swap. This prevents a runaway log file from filling the entire disk and crashing the OS.

	MBR (Master Boot Record)	GPT (GUID Partition Table)
Max disk size	2 TB	9.4 ZB (practically unlimited)
Max primary partitions	4 (use extended partition for more)	128 partitions by default
Standard	Traditional BIOS	Part of UEFI specification
Status	Being replaced	Current standard for new systems

A mount point is a directory where a partition's contents become accessible. The root partition / is always mounted. /home, /var, /tmp, /opt can each have their own dedicated partition mounted at that path.

5.3.2 fdisk — Creating Partitions

```
fdisk -l                            # List all disks and their partitions
fdisk /dev/sdb                      # Open interactive partitioner for sdb disk

# Inside fdisk interactive mode:
m                                   # Show help menu
p                                   # Print current partition table
n                                   # New partition
d                                   # Delete a partition
```

```
t                                   # Change partition type
l                                   # List partition type codes (82=swap, 83=Linux)
g                                   # Create new GPT partition table
o                                   # Create new MBR partition table
w                                   # Write changes to disk and exit
q                                   # Quit without saving
```

⚠ NOTE: fdisk alternatives: gdisk (GPT-focused) and parted. Older versions of fdisk did not support GPT — newer versions do. In interviews, if asked about GPT partitioning, gdisk or parted are also correct answers.

5.3.3 Filesystems — mkfs, mount, umount, fstab

A partition must be formatted with a filesystem before it can store data. The filesystem is the structure that organises files on the partition.

Filesystem	Description	Common Use
ext4	4th extended filesystem — default on most Linux distros	General purpose — OS, home, data
ext3	ext4's predecessor — has journaling	Older systems
ext2	Original ext — no journaling	Legacy only
XFS	High performance — good for large files	Enterprise, databases, Red Hat default
Btrfs	Copy-on-write with snapshots and RAID	Modern workloads, Fedora default
ZFS	Enterprise-grade with checksumming	Storage servers

```
# Create a filesystem
mkfs -t ext4 /dev/sdb2              # Create ext4 filesystem on partition sdb2
mkfs.ext4 /dev/sdb2                 # Same — shorthand syntax
mkfs -t xfs /dev/sdb3              # Create XFS filesystem

# Mount a filesystem (temporary — does not survive reboot)
mount /dev/sdb2 /opt                # Mount sdb2 at /opt
mount                               # Show all currently mounted filesystems
df -h                               # Show disk usage of all mounted filesystems

# Unmount
umount /opt
umount /dev/sdb2                    # Either device path or mount point works

# Swap space
mkswap /dev/sdb1                    # Format partition as swap
swapon /dev/sdb1                    # Enable swap
swapon -s                           # Show active swap devices
```

/etc/fstab — Make mounts permanent across reboots:

```
# /etc/fstab format:
```

```
# device            mount_point    fs_type  options    dump  fsck_order
/dev/sdb2            /opt           ext4     defaults   0      2
UUID=abc123-xyz      /              xfs      defaults   0      1
LABEL=opt            /opt           ext4     defaults   0      2
/dev/sdb1            swap           swap     defaults   0      0
```

`UUID` → universally unique identifier — more reliable than /dev/sdX which can change on reboot

`defaults` → use default mount options for this filesystem type

`dump column (0)` → 0 = skip backup with dump utility; 1 = include in dump backup

`fsck_order` → 0 = skip fsck check; 1 = check first (use for /); 2 = check after / (use for others)

```
# View UUIDs and labels
lsblk -f
blkid

# Set a label on an ext filesystem
e2label /dev/sdb3 opt
```

5.3.4 LVM — Logical Volume Manager

LVM adds a flexible abstraction layer between physical disks and filesystems. Instead of partitions that are fixed in size, LVM lets you create logical volumes that can grow or shrink while the system is running.

LVM Layer	What It Is	Command to Create
PV (Physical Volume)	A disk or partition initialised for LVM use	pvcreate /dev/sdc
VG (Volume Group)	A pool of one or more PVs — the storage pool	vgcreate myvg /dev/sdc
LV (Logical Volume)	A slice of the VG — what you format and mount	lvcreate -L 10G -n mydata myvg

```
# Full LVM workflow
pvcreate /dev/sdc /dev/sdd         # Initialise two disks as physical volumes
vgcreate vg_data /dev/sdc /dev/sdd # Create volume group from those PVs
lvcreate -L 50G -n lv_app vg_data  # Create 50GB logical volume named lv_app
mkfs.ext4 /dev/vg_data/lv_app      # Format the LV with ext4
mount /dev/vg_data/lv_app /opt/app # Mount it

# View LVM info
pvs                                # Summary of physical volumes
vgs                                # Summary of volume groups
lvs                                # Summary of logical volumes
pvdisplay                          # Detailed PV info
vgdisplay                          # Detailed VG info
lvdisplay                          # Detailed LV info

# Extend a logical volume online (while mounted)
vgextend vg_data /dev/sde          # Add new disk to volume group
```

```
lvextend -L +20G /dev/vg_data/lv_app  # Grow LV by 20GB
resize2fs /dev/vg_data/lv_app      # Expand the ext4 filesystem to fill LV
```

⚠ NOTE: LVM device paths follow the pattern: /dev/volume_group_name/logical_volume_name. For example /dev/vg_data/lv_app. This is what you format with mkfs and add to /etc/fstab.

5.4 Networking in Linux

```
# View network interfaces and IP addresses
ip a
ip addr show
ip addr show eth0                  # Show only eth0 interface

# View routing table
ip route

# Test connectivity
ping 8.8.8.8                       # Ping Google DNS
ping -c 4 azure.microsoft.com      # Ping 4 times then stop

# DNS lookup
nslookup azure.microsoft.com
dig azure.microsoft.com

# View listening ports
ss -tlnp                           # TCP, listening, numeric, show process
netstat -tlnp                      # Same but older command (may not be installed)

# Network config files
cat /etc/hostname                  # System hostname
cat /etc/hosts                     # Local DNS overrides
cat /etc/resolv.conf               # DNS nameserver configuration
```

`ip a` → the modern replacement for the older ifconfig command — shows all interfaces and IPs

`lo` → loopback interface — always 127.0.0.1 — used for the system to communicate with itself

`eth0 / ens3 / enp0s3` → names vary by system and driver — the actual network interface connected to the network

`ss -tlnp` → t=TCP; l=listening only; n=numeric (don't resolve names); p=show process using the port

⚠ NOTE: In Azure VMs, the network interface will typically be named eth0 or ens3 depending on the VM generation. The IP shown by ip a is the private IP. The public IP is assigned externally and visible in the Azure portal, not directly on the VM.

5.5 Phase 5 Summary Table

Topic	Key Commands / Files	Remember
Boot: BIOS	POST → bootable device	POST = hardware self-test
Boot: GRUB	/boot/grub	Loads kernel + initrd; GRUB = Grand Unified Bootloader
Boot: initrd	/boot/initrd.img	Temporary RAM filesystem for loading drivers before real root is mounted
Boot: kernel	/boot/vmlinuz	vmlinuz = compressed kernel (z suffix)
Boot messages	dmesg, /var/log/dmesg	Kernel ring buffer — check after boot for hardware issues
Targets	systemctl set-default	multi-user.target = server mode; graphical.target = desktop
Logging	/etc/rsyslog.conf	facility.severity → log file; auth goes to auth.log or secure
Log files	/var/log/	messages/syslog = general; auth.log/secure = auth; cron = jobs
logrotate	/etc/logrotate.conf	Rotates, compresses, and prunes logs automatically
Partitions	fdisk -l, fdisk /dev/sdX	MBR=2TB/4parts; GPT=9.4ZB/128parts
Filesystem	mkfs.ext4, mount, umount	Format then mount; add to /etc/fstab to persist
Disk info	df -h, lsblk -f, blkid	df=usage; lsblk=block devices; blkid=UUIDs
LVM	pvcreate→vgcreate→lvcreate	PV→VG→LV; can grow online with lvextend+resize2fs
Networking	ip a, ss -tlnp	ip a = modern ifconfig; ss = modern netstat

PHASE 6 — Interview Favourites: Advanced Topics and Shell Scripting

6.1 Special Permission Modes (setuid, setgid, sticky bit)

Beyond the standard r/w/x permissions, Linux has three special permission modes. These are heavily tested in interviews because they are security-critical and commonly misunderstood.

6.1.1 setuid — Run as File Owner

When the setuid bit is set on an executable file, the file runs with the privileges of the file's owner rather than the user who is running it. The most famous example is /usr/bin/passwd — a normal user needs to change their password, but the password file (/etc/shadow) is only writable by root. With setuid, passwd runs as root even when executed by a normal user.

```
# See if a file has setuid
ls -l /usr/bin/passwd
# -rwsr-xr-x  1  root  root  68208  /usr/bin/passwd
#    ^ s = setuid is set AND execute is set

# Add setuid to a file
chmod u+s /path/to/binary
chmod 4755 /path/to/binary         # 4 in front = setuid

# Remove setuid
chmod u-s /path/to/binary
chmod 0755 /path/to/binary         # 0 in front clears all special bits

# Find ALL setuid files on the system (security audit)
find / -perm /4000 -ls 2>/dev/null
```

`s in owner execute position (lowercase)` → setuid is set AND the owner has execute permission — file runs as owner

`S in owner execute position (uppercase)` → setuid is set BUT the owner does NOT have execute permission — unusual/meaningless configuration

`4755` → the 4 prefix enables setuid; 755 = rwxr-xr-x for the regular permissions

⚠ NOTE: Linux does NOT honour setuid on shell scripts — only on binary executable files. This is a security measure by the kernel. If you see a shell script with setuid set, it will NOT run with elevated privileges — it runs as the user who invoked it.

⚠ NOTE: setuid files owned by root are a major attack vector. If an attacker finds a setuid root file that is world-writable (chmod 4777), they can overwrite the binary with malicious code, and when any user runs it, it executes as root.

6.1.2 setgid — Run with File's Group / Inherit Group in Directory

On a file: the program runs with the group privileges of the file's group, not the group of the user executing it. The classic example is the wall command (broadcasts a message to all terminals) — it runs with the tty group privileges to write to terminal files.

On a directory: every new file or subdirectory created inside inherits the group ownership of the directory, not the creating user's primary group. This is extremely useful for shared team directories.

```
# Add setgid to a file
chmod g+s /path/to/binary
chmod 2755 /path/to/binary         # 2 in front = setgid

# setgid on a directory — all new files inherit the directory group
chmod g+s /opt/shared-project
chmod 2775 /opt/shared-project     # rwxrwsr-x
```

```
# Find all setgid files
find / -perm /2000 -ls 2>/dev/null

# Combine setuid + setgid
chmod ug+s /path/to/file
chmod 6755 /path/to/file           # 6 = 4(setuid) + 2(setgid)
```

`s in group execute position (lowercase)` → setgid is set AND the group has execute permission

`S in group execute position (uppercase)` → setgid is set but group does NOT have execute permission

`2775 on directory` → setgid directory — new files get the directory's group, not the creator's group

⚠ NOTE: setgid on a directory is commonly used in DevOps for shared deployment directories. For example, if /opt/app is owned by group devops with setgid set, any file dropped into /opt/app by any devops team member will automatically belong to the devops group — no manual chgrp needed.

6.1.3 Sticky Bit — Protect Files in Shared Directories

The sticky bit on a directory prevents users from deleting or renaming files they do not own, even if they have write permission on the directory. The classic example is /tmp — everyone can write to /tmp but only the owner of a file (or root) can delete their own files.

```
# Add sticky bit to a directory
chmod o+t /tmp/shared
chmod 1777 /tmp/shared             # 1 in front = sticky bit; 777 = rwxrwxrwx

# Check if sticky bit is set
ls -ld /tmp
# drwxrwxrwt  1  root  root  /tmp
#          ^ t = sticky bit is set AND others have execute
#          ^ T = sticky bit is set but others do NOT have execute

# Remove sticky bit
chmod o-t /directory
```

`t in others execute position (lowercase)` → sticky bit is set AND others have execute permission

`T in others execute position (uppercase)` → sticky bit is set but others do NOT have execute — unusual, mostly meaningless

`1777` → 1 = sticky bit; 777 = rwxrwxrwx

⚠ NOTE: The sticky bit is typically only meaningful on directories. You will almost always see it on /tmp and /var/tmp. In an interview, if asked "what is the sticky bit and where is it used?" — /tmp is the canonical answer.

6.1.4 Complete Special Permissions Reference

Special Bit	Octal	File Effect	Directory Effect	Set With	Find With
setuid	4xxx	Runs as file owner (not as executing user)	No effect	chmod u+s or chmod 4xxx	find / -perm /4000
setgid	2xxx	Runs with file's group	New files inherit directory's group	chmod g+s or chmod 2xxx	find / -perm /2000
sticky	1xxx	No effect	Only file owner or root can delete the file	chmod o+t or chmod 1xxx	find / -perm /1000

6.2 Shell Scripting — Writing Automation Scripts

Shell scripts are text files containing a series of commands executed by the shell interpreter. In DevOps, shell scripts automate repetitive tasks: deployments, backups, log cleanup, health checks, user account creation. Anything you can type at the command line, you can put in a script.

6.2.1 Script Structure and the Shebang

```
#!/bin/bash
# This is a comment — ignored by the shell
# Script: deploy.sh
# Purpose: Deploy the application

echo "Starting deployment..."
cd /opt/myapp
git pull
systemctl restart myapp
echo "Deployment complete."
```

`#!/bin/bash` → shebang — must be the very first line. Tells the OS which interpreter to use. #! = sharp + bang = shebang

`# comment` → any line starting with # is a comment and is ignored by the shell

`chmod 755 script.sh` → must make script executable before you can run it

`./script.sh` → run using relative path — ./ means "in this directory"

`/path/to/script.sh` → run using absolute path — works from anywhere

6.2.2 Variables

```
#!/bin/bash

# Assign a variable — NO SPACES around the = sign
APP_NAME="myapp"
APP_PORT=8080
SERVER_NAME=$(hostname)              # Capture command output into variable
TIMESTAMP=`date +%Y%m%d`            # Older backtick syntax — same result

# Use a variable with $
echo "Deploying $APP_NAME on port $APP_PORT"
echo "Server: $SERVER_NAME"

# Curly braces for clarity or when appending text
ENVIRONMENT="prod"
echo "Config file: ${ENVIRONMENT}ution.conf"   # Outputs: production.conf
# Without braces: $ENVIRONMENTution.conf would try variable named ENVIRONMENTution
```

⚠ NOTE: No spaces around = when assigning variables. APP=value works. APP = value fails. The shell treats APP = value as running a command called APP with arguments = and value.

⚠ NOTE: Variable names: letters, digits, and underscores only. Cannot start with a digit. Convention: UPPERCASE for environment variables and script-wide variables.

6.2.3 Conditionals — if/elif/else/fi

```
#!/bin/bash

# Basic if statement
if [ "$APP_NAME" = "myapp" ]
then
    echo "Correct application"
fi

# if / else
if [ "$USER" = "root" ]
then
    echo "Running as root"
else
    echo "Running as normal user: $USER"
fi

# if / elif / else
if [ $APP_PORT -eq 80 ]
then
    echo "HTTP port"
elif [ $APP_PORT -eq 443 ]
then
```

```
    echo "HTTPS port"
else
    echo "Custom port: $APP_PORT"
fi
```

Test Type	Operator	Meaning	Example
File	-e file	File exists	[-e /etc/passwd]
File	-f file	Is a regular file (not dir/symlink)	[-f /etc/nginx.conf]
File	-d directory	Is a directory	[-d /var/log]
File	-r file	File is readable	[-r /etc/shadow]
File	-w file	File is writable	[-w /tmp/data.txt]
File	-x file	File is executable	[-x /usr/bin/python3]
String	-z string	String is empty	[-z "$VAR"]
String	-n string	String is not empty	[-n "$VAR"]
String	=	Strings are equal	["$A" = "$B"]
String	!=	Strings are not equal	["$A" != "$B"]
Number	-eq	Numbers are equal	[$PORT -eq 80]
Number	-ne	Not equal	[$PORT -ne 443]
Number	-lt	Less than	[$COUNT -lt 10]
Number	-le	Less than or equal	[$COUNT -le 10]
Number	-gt	Greater than	[$SIZE -gt 100]
Number	-ge	Greater than or equal	[$SIZE -ge 100]

⚠ NOTE: Always quote variables in tests: ["$VAR" = "value"] not [$VAR = "value"]. Without quotes, if VAR is empty the test becomes [= "value"] which is a syntax error.

6.2.4 Loops

```
# for loop — iterate over a list
for SERVER in web01 web02 web03
do
    echo "Deploying to $SERVER"
    ssh $SERVER "systemctl restart myapp"
done

# for loop over command output
for FILE in $(ls /var/log/*.log)
do
    echo "Processing: $FILE"
done

# while loop — run while condition is true
COUNT=1
while [ $COUNT -le 5 ]
do
    echo "Attempt $COUNT"
    COUNT=$(( COUNT + 1 ))
done

# while loop reading a file line by line
```

```
while read LINE
do
    echo "Processing: $LINE"
done < /etc/hosts
```

6.2.5 Script Inputs, Outputs and Exit Codes

```
#!/bin/bash
# Accessing command-line arguments
echo "Script name:      $0"
echo "First argument:   $1"
echo "Second argument:  $2"
echo "All arguments:    $@"
echo "Number of args:   $#"

# Exit status — last command's result
ls /nonexistent 2>/dev/null
echo "Exit status: $?"          # $? = 0 success, non-zero = failure

# Exit the script with a status code
if [ ! -f /etc/nginx/nginx.conf ]
then
    echo "ERROR: nginx.conf not found"
    exit 1                      # Non-zero = failure
fi
echo "Config found"
exit 0                          # 0 = success

# Read input from user interactively
read -p "Enter your name: " USERNAME
echo "Hello, $USERNAME"
```

`$0` → the name of the script itself

`$1, $2, $3...` → positional parameters — arguments passed to the script

`$@` → all arguments as separate words — use in loops: for arg in "$@"

`$#` → number of arguments passed to the script

`$?` → exit status of the last command — 0 = success; non-zero = failure

`exit 0` → end script and signal success to the calling process

`exit 1` → end script and signal failure — use in error handling

`read -p` → -p shows a prompt; stores input in the named variable

6.2.6 Functions in Shell Scripts

```
#!/bin/bash
```

```
# Define a function
deploy_app() {
    local APP=$1                    # local = variable only exists inside this
function
    local ENV=$2
    echo "Deploying $APP to $ENV environment"
    # ... deployment commands ...
    return 0
}

# Call the function
deploy_app "myapp" "production"
deploy_app "api-service" "staging"
```

6.3 Advanced grep — Regular Expressions in Scripts

```
# Strip comments and blank lines from a config file
grep -Ev "^#|^$" /etc/httpd/conf/httpd.conf

# Extended regex: ^ = start of line, $ = end of line, | = OR
grep -E "error|warning|critical" /var/log/app.log

# Count error occurrences
grep -c "ERROR" /var/log/app.log

# Show 3 lines after each match (context)
grep -A 3 "CRITICAL" /var/log/app.log

# Recursive search through all files in a directory
grep -r "password" /etc/
```

`^` → in regex: caret = beginning of line

`$` → in regex: dollar = end of line

`^$` → blank line (starts and immediately ends)

`^#` → line that starts with # (a comment)

`-E` → enable extended regular expressions (allows | for OR, + for one-or-more, etc.)

`-v` → invert — show lines that do NOT match the pattern

`-Ev "^#|^$"` → combined: show lines that are not comments AND not blank — strips dead lines from configs

6.4 Tar, gzip, and Archives

```
# Create a tar archive
tar cf backup.tar /opt/myapp/
```

```
# Create a compressed tar archive (tar + gzip)
tar czf backup.tar.gz /opt/myapp/

# List contents of an archive
tar tf backup.tar
tar tzf backup.tar.gz

# Extract an archive
tar xf backup.tar
tar xzf backup.tar.gz
tar xvf backup.tar.gz              # v = verbose, shows each file as extracted

# Extract to a specific directory
tar xzf backup.tar.gz -C /opt/restore/

# Compress/decompress individual files
gzip largefile.log                 # Compresses to largefile.log.gz
gunzip largefile.log.gz            # Decompresses
gzcat largefile.log.gz             # View compressed file without decompressing
```

tar option	Meaning
c	Create a new archive
x	Extract files from archive
t	List (table of contents) of archive
z	Use gzip compression (.tar.gz or .tgz)
v	Verbose — show file names being processed
f	Specify the archive filename (must be followed by filename)

⚠ NOTE: tar does not require a hyphen before its options by tradition (tar czf = tar -czf). Both work. The f option must always be last when combining, because the filename follows immediately after f.

6.5 LAMP/LEMP Stack — Real-World Deployment Project

A LAMP stack (Linux + Apache + MySQL + PHP) or LEMP stack (Linux + Nginx + MySQL + PHP) is a complete web server platform. The course demonstrates deploying WordPress as a practical Linux system administration project.

6.5.1 LEMP Stack Installation (Ubuntu)

```
# Install Nginx
apt update && apt install -y nginx
systemctl start nginx
systemctl enable nginx            # Start on boot
systemctl status nginx
```

```
# Install MySQL
apt install -y mysql-server
mysql_secure_installation         # Harden MySQL post-install

# Install PHP-FPM and required modules for WordPress
apt install -y php-fpm php-mysql php-curl php-mbstring php-imagick php-xml php-zip
systemctl start php7.4-fpm

# Check service name using tab completion
systemctl start php<TAB><TAB>     # Shows available php service names
```

6.5.2 Configuring Nginx for PHP

```
# Test Nginx config for syntax errors before applying
nginx -t

# Reload Nginx (applies config without dropping connections)
systemctl reload nginx

# Key changes to /etc/nginx/sites-available/default:
# 1. Add index.php to the index line:
#    index index.php index.html index.htm;

# 2. Change try_files to support pretty permalinks:
#    try_files $uri $uri/ /index.php$is_args$args;

# 3. Uncomment the PHP-FPM location block
```

⚠ NOTE: nginx -t is the test command — ALWAYS run this before systemctl reload nginx. If your config has a syntax error and you reload without testing, Nginx will fail to start and your web server goes down.

6.5.3 MySQL Database Setup for WordPress

```
# Log into MySQL as root
mysql -u root -p

# Inside MySQL:
CREATE DATABASE wordpress;
CREATE USER 'wordpress'@'localhost' IDENTIFIED BY 'StrongPassword123';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wordpress'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

`CREATE DATABASE` → creates the database WordPress will use to store all its data

`CREATE USER` → creates a MySQL user account for WordPress to connect with

`GRANT ALL PRIVILEGES` → gives the wordpress MySQL user full access to the wordpress database only — not to any other DB

`FLUSH PRIVILEGES` → tells MySQL to reload its permission tables so the new grant takes effect immediately

6.5.4 WordPress Deployment

```
# Download WordPress
curl -O https://wordpress.org/latest.tar.gz

# Extract
tar xzvf latest.tar.gz

# Move to web root
mv wordpress/ /var/www/html/

# Set ownership to web server user
chown -R www-data:www-data /var/www/html/wordpress

# Find server IP for web configuration
ip a
# Then open browser to http://SERVER_IP and follow the WordPress installer
```

⚠ NOTE: chown -R www-data:www-data is critical — the Nginx web server runs as the www-data user. WordPress needs to write configuration files and updates to disk. If the files are owned by root or your own user, WordPress will get "permission denied" errors.

⚠ NOTE: Never use "admin" as your WordPress username — it is the first username every automated brute-force attack tries. Use something unique.

6.6 Phase 6 Summary Table

Topic	Key Commands	Interview Answer
setuid	chmod u+s / chmod 4755	Runs as file owner not executor. Example: /usr/bin/passwd. NOT honoured on shell scripts.
setgid	chmod g+s / chmod 2755	Runs with file's group. On directories: new files inherit directory's group.
sticky bit	chmod o+t / chmod 1777	Only owner/root can delete files in the directory. Classic use: /tmp
Find special perms	find / -perm /4000	Always check setuid files in a security audit
Shell scripts	#!/bin/bash + chmod 755	Shebang must be first line; make executable before running
Variables	VAR="value", $VAR, ${VAR}	No spaces around =; quote variables in tests
Conditionals	if [ ] then; elif; else; fi	String: = Number: -eq/-lt/-gt File: -f/-d/-e
Loops	for x in list; do; done	Use $@ to loop over all script

		arguments
Exit status	$?	0 = success; non-zero = failure
grep advanced	grep -Ev "^#|^$"	Strip comments+blank lines from configs; -r for recursive
tar	tar czf file.tar.gz dir/	c=create; x=extract; z=gzip; v=verbose; f=file
Nginx test	nginx -t	Always test config before reload — prevents outages
WordPress chown	chown -R www-data	Web server user must own web files for WordPress to write them

BONUS — Interview Quick Reference and Common Questions

Top Interview Questions and Model Answers

Question	Model Answer
What is the difference between su and su -?	su switches to another user but keeps your current environment. su - creates a full login session as that user — loads their PATH, HOME, and shell config. Always use su - unless you specifically need to keep your own environment.
What is sudo and how is it different from su?	sudo runs a specific command as root (or another user) using your own password. su switches your entire session to another user using their password. sudo is preferable for security because it requires your password, creates an audit trail, and you can restrict which commands a user can run in /etc/sudoers.
What is the difference between find and locate?	find searches the filesystem in real time — always current but slower. locate searches a pre-built index — much faster but may be up to 24 hours out of date. Use find when you need real-time results or advanced criteria. Use locate for quick searches of well-established files.
What is setuid and where is it used?	setuid causes an executable to run with the privileges of the file's owner rather than the user executing it. The canonical example is /usr/bin/passwd which runs as root so normal users can update /etc/shadow. Important: Linux does NOT honour setuid on shell scripts — only binary executables.
What does chmod 755 mean?	7=rwx for the owner (read+write+execute); 5=r-x for the group (read+execute, no write); 5=r-x for others (read+execute, no write). Commonly used for executables and directories.
What is the sticky bit and where is it used?	The sticky bit on a directory prevents users from deleting files they do not own, even if they have write permission on the directory. It is typically set on /tmp and /var/tmp to prevent users from

	deleting each other's temporary files.
What is the difference between > and >> in redirection?	> redirects stdout to a file and OVERWRITES any existing content. >> appends to the file without destroying existing content. Forgetting this difference and using > when you meant >> is a common and costly mistake.
What is a zombie process?	A zombie (defunct) process is one that has finished executing but still has an entry in the process table because its parent has not yet read its exit status. Zombies consume no resources but if the parent never cleans them up, PIDs get exhausted. Kill the parent process to clean up zombie children.
What is /dev/null?	/dev/null is a special device file — the "black hole" of Linux. Anything written to /dev/null is discarded immediately. It is used to suppress unwanted output: command 2>/dev/null discards error messages.
What is the difference between kill and kill -9?	kill sends SIGTERM (signal 15) — a polite request for the process to terminate cleanly, allowing it to save state and close files. kill -9 sends SIGKILL — the kernel terminates the process immediately with no cleanup. Always try SIGTERM first; use SIGKILL only if the process is frozen.
What is umask?	umask (user mask) controls the default permissions of newly created files and directories by subtracting from the system defaults (666 for files, 777 for directories). A umask of 022 results in files with 644 (666-022) and directories with 755 (777-022) permissions.
What are runlevels vs. systemd targets?	Runlevels (0-6) are the traditional init system way of defining system states. systemd targets are the modern equivalent: poweroff.target=0, rescue.target=1, multi-user.target=2-4, graphical.target=5, reboot.target=6. Use systemctl set-default and systemctl isolate to change targets.
What is LVM and why use it?	LVM (Logical Volume Manager) adds a flexible abstraction layer between physical disks and filesystems. It allows: filesystems that span multiple disks, online resize of volumes without reformatting, online data migration, and human-readable device names. The hierarchy is: PV (physical disk) → VG (pool) → LV (logical partition).
What is the /etc/fstab file?	/etc/fstab is the filesystem table — it controls which devices are mounted at boot, where they are mounted, what filesystem type they use, and with what options. Without an entry in fstab, a manually mounted filesystem will not survive a reboot.

Study Complete. Document ready for review and OneNote import.